

Fundamental Git & GitHub untuk Kolaborasi Tim



Disusun oleh:

Muhammad Idris S.Tr., M.Tr.Kom



Tujuan

- Mahasiswa mampu menggunakan Git dan GitHub untuk mengelola perubahan kode secara individu maupun tim mulai dari membuat repository, melakukan commit, bekerja dengan branch, mengajukan Pull Request, hingga menyelesaikan merge conflict **sebagai dasar praktik version control.**





Teori Singkat

- **Git** adalah sistem version control terdistribusi yang mencatat setiap perubahan pada kode sebagai riwayat (history) yang bisa ditelusuri, dibandingkan, dan dikembalikan.
- **GitHub** adalah platform hosting berbasis Git yang menambahkan lapisan kolaborasi: pull request, code review, issue tracking, project board, hingga otomasi (GitHub Actions).





Konsep Utama

- **Repository (repo)**: wadah proyek beserta seluruh riwayat perubahannya.
- **Remote**: salinan repo yang berada di server (GitHub).
- **Commit**: snapshot perubahan disertai pesan penjas, membentuk jejak audit.
- **Branch**: jalur pengembangan paralel agar perubahan tidak langsung mengganggu kode utama (main).
- **Pull Request (PR)**: mekanisme mengusulkan penggabungan perubahan disertai proses review, menjadi titik kontrol kualitas dan keamanan sebelum kode masuk ke branch utama. Biasa juga disebut dengan istilah Merge.
- **Merge conflict**: situasi ketika dua perubahan bentrok pada bagian kode yang sama dan harus diselesaikan manual.





Instalasi Lab/Environment

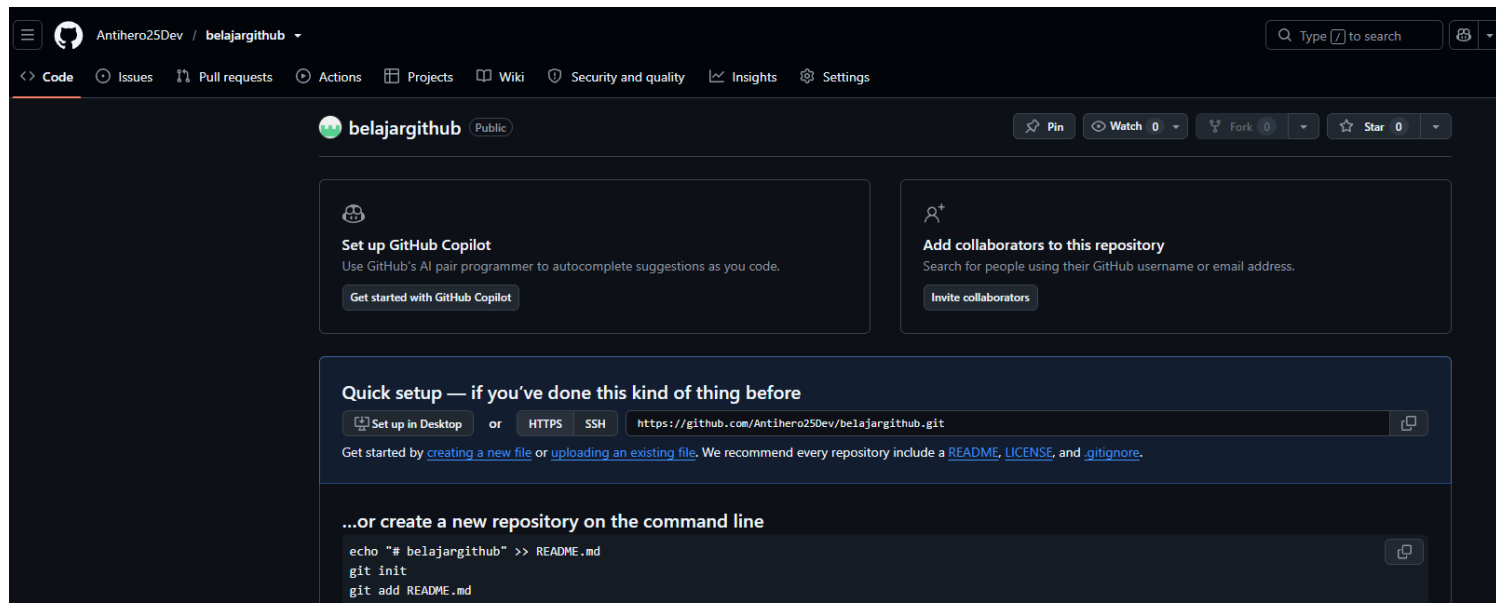
- Buat akun di <https://github.com>
- Install Git sesuai OS.
- Cek versi Git: **git --version**
- Buat folder proyek (misal: E:\belajargithub)
- Open folder di VSCode > File > Open Folder > Pilih E:\belajargithub





Lab 1: Menyiapkan Folder Proyek

1. Buat repository baru di GitHub lewat web (New repository). Misal: belajargithub
2. Pilih public mode
3. Centang opsi untuk membuat README.md





Personal Access Token di GitHub

- Buat PAT: Settings > Developer settings > Personal access tokens > Tokens (classic) > Generate new token (classic) > Centang repo > Klik Generate
- Simpan token tersebut di notepad, karena akan dipakai sebagai password saat git push pertama kali (Git akan meminta autentikasi).

Note: Saat melakukan git push pertama kali, akan muncul terminal untuk memasukkan Password, pada tahap ini jangan masukkan password biasa akun GitHub, melainkan paste-kan kode PAT yang telah disimpan tadi.





Identitas Khusus di Folder Proyek

- Buka terminal di VScode: **Terminal > New Terminal**
- Inisialisasi Git di folder dengan perintah: **git init**
- Jalankan perintah berikut:

```
git config --global user.name "Nama Akun"
```

```
git config --global user.email "email-akun@example.com"
```
- Untuk memastikan berhasil, Jalankan **git config --global user.name** dan **git config --global user.email**

Sama dengan
akun github anda

Note: Jika komputer sudah pernah terhubung ke Github, maka tidak perlu melakukan tahapan ini

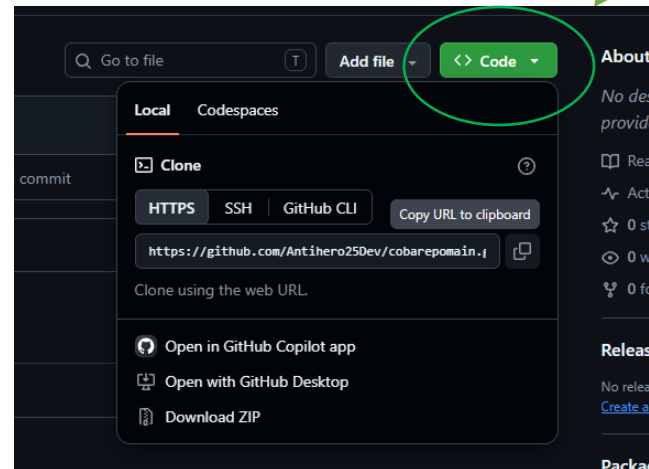




Hubungkan folder ke repositori GitHub

Jalankan perintah berikut:

```
git remote add origin https://github.com/UsernameAkun/belajargithub.git  
git remote -v untuk verifikasi
```





Lab 1 (pull&merge)

1. Ubah branch menjadi main (agar sama dengan web) dengan perintah:

`git branch -M main`

2. Ambil (fetch) dan gabungkan (merge) perubahan terbaru dari cabang (*branch*) main yang ada di repositori online ke lokal dengan perintah:

`git pull origin main`

Note: Gunakan perintah ini secara berkala sebelum mulai menulis kode baru atau sebelum membuat *push* baru untuk memastikan kode di komputer lokal tetap *up-to-date* dengan repo online (tim).





Lab 1 (push)

1. Ubah branch ke main (agar sama dengan web) dengan perintah **git branch -M main**
2. Buat file baru dengan nama **app.py**. Isikan dengan kode berikut: `print("HELLO WORLD")`
3. Tambahkan file ke staging area dengan perintah: **git add .**
4. Buat commit pertama: **git commit -m "initial commit"**
5. Push ke GitHub: **git push origin main**
6. Masukkan **PAT** yang sudah disimpan tadi
7. Verifikasi dengan membuka repository di GitHub, pastikan file app.py muncul.





Lab 2: Branching dan Commit History

1. Buat branch baru: `git checkout -b fitur-login`
2. Buat file **login.py** dan isikan dengan `print("INI FITUR LOGIN")`
3. Cek perubahan: `git status`
4. Staging dan commit perubahan pertama: `git add .`
5. Commit dengan: `git commit -m "menambahkan fitur login"`
6. Kirim ke github dengan perintah: `git push -u origin fitur-login`





Lab 2 (lanjutan)

6. Ulangi edit file `app.py` dan tambah baris print nama anda
7. Jalankan kembali **`git add .`** lalu **`git commit -m "menambahkan nama saya"`**
8. Push branch ke GitHub: **`git push -u origin fitur-login`**
9. Lihat riwayat commit ringkas: **`git log --oneline`**
10. Lihat riwayat commit lengkap (author, tanggal): **`git log`**
11. Bandingkan dua commit: **`git diff <commit-id-1> <commit-id-2>`**





Lab 3: Pull Request dan Code Review

1. Pastikan branch fitur sudah di-push (lihat Lab 2, langkah 5).
2. Buka **GitHub** > **tab Pull Requests** > **New pull request**.
3. Pilih base: main, compare: fitur-login, lalu klik **Create pull request**. Isi judul dan deskripsi PR.
4. klik **Merge pull request**.
5. Sinkronkan branch main lokal dengan hasil merge:
 - `git checkout main`
 - `git pull origin main`

(Optional) Jika ada kolaborator, Tambahkan reviewer lewat panel di sisi kanan PR. Reviewer memberi komentar di baris kode langsung lalu submit review (Approve/Request changes).





Lab 4: Simulasi Merge Conflict

1. Buat file baru dengan nama **checkout.py**
2. Isikan dengan: **total = 100_000**
3. Commit & Push:
git add .
git commit -m "set total checkout"
git push origin main





Buat & Push Branch add-diskon

- Buat branch baru dari terminal: `git checkout -b add-diskon`
- Edit file checkout.py di VS Code: `total = 100_000 * 0.5`
- Commit & push ke GitHub:
 `git add .`
 `git commit -m "setengah harga untuk semua"`
 `git push -u origin add-diskon`





Ubah main di Lokal

- Agar terjadi bentrokan saat PR nanti, ubah lagi baris tersebut di branch main: **git checkout main**
- Edit **checkout.py** di VS Code menjadi: **total = 100_000 * 0.2**
- Commit dan push ke GitHub:
git add .
git commit -m "semua dapat diskon 20%"
git push origin main





Selesaikan Conflict di PR GitHub

1. Buka GitHub, buat **Pull Request** dari fitur-diskon ke main.
2. GitHub akan menampilkan pesan merah "**Can't automatically merge**". Klik **Create pull request**.
3. Klik tombol **Resolve conflicts** di halaman PR.
4. Accept current change (pakai versi branch), Accept incoming change (pakai versi main) atau Accept both changes (pakai keduanya)
5. Klik **Mark as resolved** → **Commit merge**
6. Setelah tidak ada konflik, klik **Merge pull request** → **Confirm merge**.
7. Masuk kembali ke branch fitur-diskon: `git checkout fitur-diskon`
8. Pull kode terbaru dari main: `git pull origin main`



Terima Kasih

Ada Pertanyaan?